

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Fixed (→ Fixed.md)	9
Bug	In progress	2
Bug	Operator-blocked	1
Bug	Queued	16
Bug	Ready for testing	3
Task	Completed (→ Fixed.md)	1
Task	In progress	9
Task	Operator-blocked	3
Task	Queued	21
Task	Ready for testing	2
TOTAL		67

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-ho SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Ja + compose env-forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Aut commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape bac
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

ATM ID	Type	Status	Severity	Description
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/m service v1.0.0 readiness ledger (closes GA-10)
BOB-087	Task	Completed (→ Fixed.md)	High	RD2-20: Wire docs_chain / commit-sea sync hook per §11.4.106(F) so DB write cannot land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tra Items + Status Documents table row-completeness (GA-07 remainder)
BOB-090	Task	In progress	Medium	RD2-25: HelixQA Challenge entry exer all three start.sh subcommands end-to against real compose stack
BOB-093	Task	In progress	High	RD2-28: Live compose bring-up + veri. rutracker ReDoS regex bounds deploy container (closes runtime half of GA-12)
BOB-094	Task	In progress	Medium	RD2-29: Author tests/stress/ test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/ test_scheduler_hooks_sse_stress_chaos for Go-side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is -profile go parity still release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge

ATM ID	Type	Status	Severity	Description
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source if
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation gold bad fixture for the rate-limit detector
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #
BOB-129	Bug	Fixed (→ Fixed.md)	Medium	Potential production slowapi/starlette flagged by Task 105 subagent
BOB-131	Bug	Fixed (→ Fixed.md)	Medium	qbittorrent-proxy podman common crash pre-existing, surfaced during BOB-129 investigation
BOB-135	Bug	Ready for testing	Low	Test isolation: test <code>list_hooks_after_create</code> fails in bulk suite (Permission denied / config)
BOB-137	Bug	In progress	High	Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)
BOB-141	Task	Queued	Low	CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds 7187 — doc contradicts the Dockerfile
BOB-143	Bug	Queued	Medium	Orphaned <code>.worktrees/</code> dirs (46M, unresolvable <code>gitdir</code>) pollute gate scan and manufacture false BOB-126-class findings
BOB-144	Bug	Fixed (→ Fixed.md)	Low	<code>/theme/stream</code> calls the <code>disconnect</code> procedure unguarded — fail-closed but via an uncatched traceback, inconsistent with the two S generators
BOB-145	Bug	Fixed (→ Fixed.md)	High	Fix the 7187 wedge: offload and/or merge <code>Deduplicator.merge_results</code> so $O(N^2)$

ATM ID	Type	Status	Severity	Description
				regex work stops blocking the asyncio loop
BOB-146	Bug	Fixed (→ Fixed.md)	High	Constitution §11.4.252 detector undercounts by 29% (30 vs 42 AST gro truth) — 4 distinct blind spots make its output a floor, not a census
BOB-148	Bug	Fixed (→ Fixed.md)	Medium	Standing red unit test nothing tracked test_no_credentials asserts has_session False, gets True — real defect or non-hermetic test, undecided
BOB-149	Bug	Queued	Low	Managed-plugin count diverges 43/42/ across constitution, CLAUDE.md and the README badge; the badge is hand-maintained and unguarded
BOB-150	Task	Queued	Low	pre_build_verification.sh invariant label read N/44 but only 35 invariants are labelled
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named gro with no implementation, and 24 of 36 scripts have no companion doc
BOB-152	Bug	Queued	Medium	Constitution sweep walks vendored third party code: 82% of one gate's 38,291 findings come from submodules/
BOB-153	Bug	Fixed (→ Fixed.md)	Medium	Go profile cannot build: go.mod requires 1.26.2 but the Dockerfile builder is golang:1.23-alpine
BOB-154	Bug	Ready for testing	Medium	Host venv and production container run different starlette versions (1.4.1 vs 1.0)
BOB-155	Bug	Fixed (→ Fixed.md)	High	workable-items diff reports 'DB and Markdown are in sync' having opened Markdown files when -issues/-fixed are omitted
BOB-156	Bug	Queued	Medium	BOB-145 event-loop regression guard is load-sensitive and flaky: 8786ms under load vs a 900-1500ms ceiling calibrated on quiet host
BOB-157	Bug	Fixed (→ Fixed.md)	High	Our own BOB-137 stall watchdog can segfault the merge service: faulthandler.dump_traceback(all_threads=True) hits unpatched CPython 3.12 defect
BOB-158	Bug	Queued	High	tests/conftest.py cannot run on the production interpreter: binds asyncio.events.get_event_loop_policy, 3.13+ private API, while production is 3.12.13
BOB-159	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:01:17Z Reported-By: T041 independent review (IMPORTANT-2), partially remediated (the report, verbatim): The -recreate was fixed this round: <pre>run_ownership_precondition -> stack_up -> run_ownership_repair -> stack_up, repair walks a tree no container can walk to. That ordering is now asserted behaviourally by three new checks in test/unit/test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 OK). The WARM path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but the stack is ALREADY running, containing write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records complete of a tree that is not, and those stragglers are never repaired without a manual -force BOUNDED, not dismissed: after any successful pass the marker is valid for scope fingerprint and the repair short-circuits without walking (marker_is_valid so the window requires a stale-or-absent marker AND a live stack together. Deciding a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather hidden: the clean guard is a cheap marker only probe mode on scripts/ownership_repair.sh that answers has-scope-been-repaired without walking the tree. The existing -dry-run cannot serve deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-stack probe would walk the tree twice on every start. Adding that mode requires editing ownership_repair.sh, which another staff was actively editing during this round; duplicating marker_is_valid into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misle</pre>

ATM ID	Type	Status	Severity	Description
BOB-160	Task	Queued	Medium	comment at the warm-start call site (w claimed the gate runs before any conta writes) has been corrected to state this boundary honestly. Affected scope / fix scope manifest: start.sh (warm-start dispatch, run_ownership_gate call site scripts/ownership_repair.sh (marker fa path) Reproduction / context: 1. Ens the stack is UP. 2. Change config/ owned_paths.yaml so the scope finger changes (this re-arms the marker by d data-model E2). 3. Run a warm ./start. -recreate). 4. The ownership repair wa and chowns the declared tree while containers are still running and able to write into it. Acceptance criteria: A warm ./start.sh cannot complete an ownership repair while a container tha writes to a declared location is running. Either the repair is deferred with an actionable refusal naming -recreate, o stack is quiesced for the walk. Asserte behaviourally in tests/unit/ test_start_reload_recreate.sh alongside existing PRECONDITION_BEFORE_DOWN REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with paired 1.1 mutation that kills it. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:02:48 Reported-By: AI What (the report, verbatim): Independent §11.4.209 rev (IMPORTANT-3) found two of the featur strongest automated checks were neve executed by any runner: (1) tests/ ownership/ test_container_writes_owned_files.py - §11.4.115 RED-turned-regression-guar FR-011/FR-007, proven via a docker- compose.yml revert mutation — was m out of tests/integration/ (commit 58d34 to escape an autouse fixture hang) but runner (ci.sh, test.sh, run-all-tests.sh) ever extended to cover tests/ownership tests/pre_build/test_.sh, the §1.1 paired mutation meta-tests for the scripts/ pre_build/check_cm_.sh gate family, wa executed by NOTHING — self-reported honestly in commit 04742d7's own me ('STILL OPEN (reported, not fixed): tes

ATM ID	Type	Status	Severity	Description
BOB-161	Task	Queued	High	pre_build/ is executed by NOTHING') I never tracked as a workable item (a §11.4.197 loss-of-requirements gap) and never wired into scripts/ pre_build_verification.sh invariant 30, globbed only tests/unit/test_.sh. Affect scope / file-scope manifest: ci.sh; sc pre_build_verification.sh invariant 30 (BASH-UNIT-TESTS-EXECUTED) Repr ion / context: grep -rn test_container_writes_owned_files ci.sh test.sh run-all-tests.sh scripts/ returned runner hits (only docs/specs mentioned path); grep in scripts/ pre_build_verification.sh showed invar 30's for-loop globbing only tests/unit/ test_.sh, never tests/pre_build/test_.sh Acceptance criteria: ci.sh gains a run gated pytest tests/ownership/ stage (sh honestly, rc=0, when no podman/docke present); scripts/pre_build_verification invariant 30's glob covers both tests/u test_.sh and tests/pre_build/test_*.sh, verified by an extracted standalone run the invariant's exact block reporting a zero RAN count that includes files from directories. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:31:12 Reported-By: BOB-092 remediation, residual finding What (the report, verbatim): §11.4.69 names CM-NO-FA OPEN-SKIP as one of three mandatory build gates: it audits sink-side probe h and FAILs if any code path converts an empty or unreachable response into a counting SKIP for a feature class with sink-side probe. This project does not l it. Why this matters now rather than in abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this from tests/e2e/test_live_stack_evidenc — the nnmclub SKIP-on-404 (already removed at 7baef2b) and a surviving s at test_iporrents_is_authenticated_in_se that skipped on 'not authenticated and status != success' while asserting 'treas as transient outage'. That assertion wa false for rejected credentials

ATM ID	Type	Status	Severity	Description
				(upstream_http_403) and for a broken container (plugin_env_missing), both of which are definitive product failures that the product already classifies via error_type. Two fail-opens of one class, found one time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime did not survive either, an agent reading code did. The remediation's own guards are AST-structural and lethal under three discriminating mutations, but they LIVE THE FILE THEY GUARD, so deleting the file evades them entirely. A pre-build guard the durable home because it audits the corpus rather than one file. Honest boundary (§11.4.6): this item does NOT claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CI has no corpus-wide detector, so the new instance in a different file is invisible and Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard) Reproduction / context: grep -rl 'CM-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Control needs the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)(b)). Acceptance criteria: A scripts/pre_build/gate named CM-NO-FAIL-OPEN-SKIP is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converted to an empty/unreachable/error response. PASS-counting SKIP for a feature class HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FALSE-with-carriage honest topology/geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/

ATM ID	Type	Status	Severity	Description
BOB-162	Task	Queued	Medium	§11.4.201(1). Paired §1.1 mutation making the gate FAIL before the gate is trusted. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:35:02 Reported-By: BOB-106/BOB-107 remediation, residual What (the report verbatim): The TESTS for both guards now executed: pre-build invariant 30's was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after: the glob expanded to 27 suites with 0 under tests/hooks which existed there; it now expands to 30 with 1 under tests/hooks. All three pass. But the GUARDS themselves are still invoked for nothing. check-brief-inputs.sh is honest conductor-run rather than automatic: a brief's required-input list lives in free-prose that the Agent tool's structured does not expose, so an automatic prompt scraping gate would itself be an unproductive pattern-match. Its seam is the dispatch procedure, and that is a documentation habit change, not a hook. unattributed commit-guard.sh is different and is where the item exists. Run against real history it names 14 violating commits since the 1 tag (and 21 bare Auto-commit subjects across all refs). Wiring it into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, on pre-existing debt nobody can fix in the moment. That is precisely the 11.4.224(E) brown adoption shape, and 11.4.66/11.4.122 the choice with the operator, not with the agent inventing a ratchet. The options stated so the decision is a decision and default: (a) immediate hard floor – the refuses until all 14 are attributed; (b) a time monotone-decrease ratchet (the 11.4.135 pattern) – snapshot 14 as the baseline, the count may fall and may not rise, so day one is green and the disease cannot spread; (c) changed-commits-or-gate only commits created from now on with a scheduled deadline for the history 14; (d) report-only for a stated period, escalate. Recommendation, with the reasoning rather than just the pick: (b) the pattern this constitution already uses

ATM ID	Type	Status	Severity	Description
				exactly this situation, it makes the deb visible and bounded immediately, and cannot be satisfied by deleting the gua name (11.4.227 closes that gaming channel). But it is the operator's call a this item is not closed until that answe recorded. Honest boundary (11.4.6): th item does NOT claim the 14 commits a harmful in themselves - it claims their ATTRIBUTION is absent and that noth currently prevents the 15th. Affected scope / file-scope manifest: scripts/unattributed-commit-guard.sh; scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/comm push-all.sh Reproduction / context: l guards run correctly by hand and are covered by passing tests (9/9 and 15/1 each proven non-bluff against a no-op Neither is invoked by scripts/pre_build_verification.sh or scripts/comm push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard no execution seam is not coverage; registration is not coverage). Accepta criteria: Each guard is invoked by a n seam, OR carries a registered deferral pointing at this item. For unattributed-commit-guard.sh specifically, the opera has answered the adoption question be and the answer is recorded as consum DATA - never an invented ratchet. Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:41:08Z Reported-By: BOB-114 remediation, p existing defect surfaced by BOB-111 la a real limiter What (the report, verbatim): PRE-EXISTING, NOT INTRODUCED - and proven so rather asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD, and the fa reproduces against the unmodified HE script. What changed is not the challer but the SYSTEM: BOB-111 appears at partially delivered, because :7187 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s. This is a 11.4.248-c corrosion risk and that is why it is filed rather than left as a footnote:
BOB-163	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				run_all_challenges.sh will now fail INTERMITTENTLY, and an intermittent trains everyone to re-run until green, at which point a real regression is dismissed 'probably the flaky rate-limit one'. THE DECISION, which is an operator call u 11.4.66 and which I deliberately did N invent: Does a 429 from a WORKING l count as the sibling endpoint being RESPONSIVE? (a) YES - a 429 proves endpoint is alive and correctly protect itself. Assertion (c) accepts 2xx OR 429 only a connection failure / 5xx / timeout counts as degraded. Risk: a genuinely wedged endpoint that happens to answer 429 would pass. (b) NO - keep requiring 2xx, but make the challenge limiter-aware drain or wait out the window before the sibling probe (retry-after is served, so budget is knowable rather than guesser) (c) Probe siblings on a path the limiter exempts (a healthz-class route), so the isolation question is asked without spe the bucket. Recommendation with reasoning, not just a pick: (a) compose with a bounded liveness check - a 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service, and (b) makes the challenge s and couples it to a window value that v drift. But the semantics of 'responsive' are a product judgement, so the answer recorded, not assumed. RELATED, stat fact rather than folded in: the detector counts 429 only, not 503, though the s header says '429 (or equivalent)'. Coun 503 would collide with the crash detect 5xx tally. Worth deciding when BOB-11 lands limiters on :7185 and :7189. Affe scope / file-scope manifest: challenge scripts/ddos_resilience_challenge.sh assertion (c) cross-endpoint isolation; run_all_challenges.sh which invokes it Reproduction / context: Assertion (c) requires a sibling-endpoint probe to ar ^2 (a 2xx). :7187 now enforces a real l (measured live: x-ratelimit-limit: 120, x-ratelimit-remaining: 86, retry-after: 45 server: uvicorn). A full challenge run s roughly 250 requests to :7187, so sibli

ATM ID	Type	Status	Severity	Description
BOB-164	Bug	Queued	Medium	probes fired during the OTHER endpoints land on an exhausted bucket and receive 429 - which assertion (c) score 'endpoint degraded'. Timing-dependent run measured PASS=5 FAIL=2; the UNMODIFIED HEAD script against the same live stack ~30s later measured PASS=7 FAIL=0 SKIP=2. Acceptance criteria: The operator has answered the classification question below, the answer recorded as consumer DATA, and assertion (c) implements it. The challenge then produces the same verdict across 3 consecutive runs against a rate-limited stack (11.4.50 deterministic consistency) and the fix ships a paired 1.1 mutation proving assertion (c) still catches a genuinely degraded sibling endpoint - narrowing it must not blind it (11.4.20). Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:56:53Z Reported-By: BOB-110 UX-class coverage discovered by the new axe-core suite on first live run What (the report, verbatim) This is a REAL user-facing defect, not a tuning artifact, and it was found by the automated regime rather than by a human squinting at the page - which is exactly 11.4.238 posture the project is aiming for. The static-grep oracle would NOT have found it. Measured: the served root shows empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct. Severity reasoning, stated rather than assumed: this is user-visible and affects primary dashboard heading, but it degrades legibility rather than breaking functionality. The surface is operator-facing rather than public. Medium, not High. The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests currently reports 1 failed, 16 passed; this is this defect. Anyone reading a red UX should read it as this item, not as flaking, and when this is fixed the suite goes full green, which is the signal that it is closed.

ATM ID	Type	Status	Severity	Description
BOB-165	Bug	Queued	High	HONEST SCOPE LIMIT (11.4.6): only dashboard landing view was scanned. jackedt/* sub-routes and the ng-serve-h:4200 route set were NOT audited – the commands to close both are recorded docs/testing/ux_accessibility.md. So the item’s 21 nodes are a floor, not a total. Affected scope / file-scope manifest Angular dashboard served at http://localhost:7187/ (same compiled SPA as frontend/); production component CSS test files Reproduction / context: Run tests/ux/test_live_dashboard_accessibi against the running merge service. axe v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color contrast violations on 21 nodes. Measure pairs: .brand / h1 text #9d001e on background #3c3f41 = 1.43-1.62:1 (WCAG AA large-text floor is 3:1); tagline #80 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1). Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component rather than by relaxing the assertion of excluding the rule (11.4.120: reconcile the correct mechanism, never weaken check). The existing tests/ux/ suite is guarded and already fails today, so the R captured – closure requires it flipping GREEN against the live surface, which runtime-class evidence per 11.4.226. Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T20:00:08Z Reported-By: surfaced while capturing closure evidence for BOB-092; diagnosed rather than worked around What (the report, verbatim): This is a STALE-AFTER-INTERPRETER-UPGRADE defect, not a missing package. The package is installed; its compiled half is built for the previous interpreter. That distinction matters because ‘pip install rpds-py’ still advice can appear to succeed while leaving the 313 artefact in place. WHY IT WAS NOTICED EARLIER, stated as fact: the paths that DO work all avoid system python3. ci.sh selects an interpreter through _select_python; the long-runni

ATM ID	Type	Status	Severity	Description
				suites in this session ran <code>.venv/bin/pyt</code> <code>-m pytest</code> and passed. So the automate regime is green while the DOCUMENT operator command is broken - an 11.4 shaped gap, since the escape was found an agent capturing evidence rather than the regime. WORKAROUND (measured theorised): <code>PYTEST_DISABLE_PLUGIN_AUTOLOA</code> with the needed plugins passed explici <code>pytest_timeout ...</code>) avoids the schemat autoload that drags in <code>jsonschema -></code> referencing <code>-> rpds</code>. That is a workaro NOT the fix: it silences the import path rather than repairing the install, and it surprise the next person who runs the documented command verbatim. RELA and worth deciding together rather than twice: BOB-154 wants <code>.venv</code> rebuilt on to match the container (which runs CP 3.12.13 while both system and venv ru 3.14.6). There are therefore THREE interpreters in play. Whoever rebuilds venv should confirm <code>rpds</code> resolves for t TARGET interpreter afterwards, or thi same class reappears one directory ov HONEST BOUNDARY (11.4.6): this ite does NOT claim any test is wrong or a product code is broken. The product a venv-run suites are unaffected. What is broken is the documented entry point. Affected scope / file-scope manifest user site-packages (<code>~/.local/lib/python</code> <code>packages/rpds/</code>); every CLAUDE.md- documented <code>'python3 -m pytest ...'</code> invocation; any tooling that uses syste <code>python3</code> rather than <code>.venv/bin/python</code> Reproduction / context: <code>python3 -m</code> <code>tests/e2e/test_live_stack_evidence.py -</code> <code>-import-mode=importlib -></code> <code>ModuleNotFoundError: No module nam</code> <code>'rpds.rpds'</code>, raised during collection via <code>schemathesis plugin autoload -></code> <code>jsonschema -> referencing. _core -> rp</code> MEASURED root cause: system python 3.14.6, but <code>~/.local/lib/python3/site-</code> <code>packages/rpds/</code> ships <code>rpds.cpython-313</code> <code>x86_64-linux-gnu.so</code> - an extension bui 3.13. <code>rpds/init.py</code> does <code>'from .rpds imp</code> and a 313-tagged <code>.so</code> is not importable

ATM ID	Type	Status	Severity	Description
				3.14, so the submodule genuinely does exist for this interpreter. The venv is CORRECT and unaffected: <code>.venv/lib64/python3/site-packages/rpds/ ships rpds.cpython-314-x86_64-linux-gnu.so</code> and <code>'venv/bin/python -c import rpds'</code> succeeds. Acceptance criteria: <code>python3 -m pytest tests/unit/ -v -import-mode=imp</code> - the command <code>CLAUDE.md</code> document reaches collection without <code>ModuleNotFoundError</code>, OR <code>CLAUDE.md</code> corrected to document the supported runner explicitly. Either way the documented command and the working command agree (11.4.99: a guide that misleads is the documentation-layer equivalent of a PASS-bluff). WORKAROUND: SIDE EFFECTS MEASURED 2026-08-27: the recipe is not free, and the item previously implied it was. <code>PYTEST_DISABLE_PLUGIN_AUTOLOAD</code> disables ALL plugin autoloader, not just the broken one, so anything the suite silently relied on must be re-enabled by hand: BREAKS 5 pre-existing env-dependent tests in <code>tests/unit/test_plugin_rutacker.py</code> - <code>TestConfig::test_default_mirrors</code>, <code>test_env_mirrors_override</code>, <code>test_get_env_with_default</code>, <code>test_get_mirrors_from_env_empty</code>, <code>test_get_mirrors_from_env_whitespace</code> which need an autoloader plugin. PROVEN not caused by any in-flight change: running HEAD's OWN copy of that file under identical flags produces the same failures. - It makes <code>-timeout=60</code> (set in <code>pyproject.toml</code>) an UNKNOWN ARGUMENT unless <code>-p pytest_timeout</code> is passed explicitly, so <code>pytest.mark.timeout</code> is silently inert under the naive recipe — a test believed to be time-bounded is not. - With autoloader and only schemathesis disabled (<code>-p no:schemathesis</code>), that same file is 96% green. CONSEQUENCE FOR ANYONE USING THE WORKAROUND: <code>-p no:schemathesis</code> alone is strictly better than blanket autoloader-disabling where it suits because it removes only the broken plugin. Where the blanket form IS used, <code>-p pytest_timeout</code> must be added or timeout

ATM ID	Type	Status	Severity	Description
BOB-166	Bug	Queued	High	are inert, and the 5 env-dependent fail must be recognised as workaround art rather than filed as defects. That last p is the real risk: the workaround manufactures failures that look exactly product defects. This does not change root cause (the venv's rpds ships a cpython-313 ABI tag CPython 3.14 can import) or the fix (rebuild the venv — BOB-154). It documents that the interi recipe has a blast radius, so nobody re workaround artifact as a regression. WHAT. §11.4.19 requires closure migr to be ATOMIC: a resolving item moves Fixed.md, DISAPPEARS from Issues_Summary (open-only) and APPE in Fixed_Summary (closed-only). Ten r violate all three properties right now — carry a terminal status yet current_location='Issues', so they rend docs/Issues.md and are listed in docs/ Issues_Summary.md with a status that literally reads '(→ Fixed.md)', while appearing 0 times in Fixed_Summary.m Any reader of the canonical open track told these are open work. MANIFEST (measured 2026-08-21): BOB-087 (Completed), BOB-129, BOB-131, BOB BOB-145, BOB-146, BOB-148, BOB-15 BOB-155, BOB-157 (all 'Fixed (→ Fixed.md)'). Confirmed rendering: BOB-155/087/157 each grep 1 in Issue 0 in Fixed.md, present in Issues_Summary.md, 0 hits in Fixed_Summary.md. ROOT CAUSE (reproduced at runtime, not inferred). 'close' performs the atomic migration REQUIRES -evidence. 'update -status' any §11.4.15 closed-set value with NO evidence and NO migration — and it k the location, because it prints it. On a of the real DB: \$ workable-items updat BOB-065 -db repro.db -status 'Fixed (Fixed.md)' update: BOB-065 updated i Issues (status=Fixed (→ Fixed.md), type=Task) \$ sqlite3 repro.db 'SELEC atm_id,status,current_location ...' BOB Fixed (→ Fixed.md) Issues So the seam is supposed to be the ONLY closure pa (close, evidence-gated per §11.4.146(D

ATM ID	Type	Status	Severity	Description
BOB-167	Bug	Queued	Medium	has a parallel unguarded path that reads the same status while skipping BOTH the evidence requirement and the migration WHY NOTHING CAUGHT IT (§11.4.23 coverage-escape audit). Three standing checks stay GREEN on a DB holding the forbidden row, verified on the poisoned from the repo root so no cwd artifact is involved: (1) ‘validate’ → ‘OK — 164 it all invariants satisfied’ (it has no status↔location coherence invariant); ‘diff’ → ‘in sync’ (DB and Markdown agree — on the WRONG state; agreement is correctness); (3) CM-CLOSURE-SEAM BINDS CHECK A → PASS (it flags NON-terminal rows whose id appears in a write commit; these rows are terminal, so they are outside its predicate by construction. This was found by reading a status tally by the automated regime — a §11.4.23 discovery-channel escape, which is itself a defect of equal standing to the mis-location rows. ACCEPTANCE. (a) ‘update’ REFUSES a terminal status and names ‘close’ as the correct path, with a paired §1.1 mutation proving the refusal (removing the guarantee must make the mutation pass). (b) ‘validate’ grows a status↔location coherence invariant that FAILS on the forbidden state with a golden-bad fixture and a negative control (a legitimately terminal row in must NOT fire — §11.4.201(1)). (c) The existing rows are drained to Fixed with class-matched evidence per row, or, when row’s evidence cannot be produced, honestly re-opened rather than migrated a bare assertion. (d) Honest boundary: closes the update-path hole and the detection gap; it does not claim every historical status write was evidence-based NOT CLAIMED. No fix is implemented on this filing. The 10 rows are untouched; draining them is acceptance (c) and each needs its own evidence, not a bulk UPD WHAT. download-proxy exposes two Sent-Events routes. They are the same expensive class — long-lived connections that hold a worker and a generator for lifetime — but only one is rate-limit class routes.py:801 @router.get('/search/status/

ATM ID	Type	Status	Severity	Description
				<pre>{search_id}') @_rl('sse_stream') <- cla routes.py:150 @router.get('/theme/stre async def stream_theme(...) <- NO limi decorator _rl(cls) resolves to limiter.limit(limit_for(cls)), so /search/s is bound to the sse_stream class while theme/stream falls through to the application default. Measured live on t running stack: /api/v1/theme/stream re x-ratelimit-limit 120. PROVENANCE + CORRECTION WORTH KEEPING (\$11 This was surfaced by the BOB-109 sca agent, whose report framed it as: 'sse_stream_limit_decorator is defined imported/applied nowhere ... SSE falls through to the 120/minute default: 24x protected'. That framing is WRONG an NOT filed as given. The agent searched one symbol NAME; the wiring uses a different mechanism (@_rl('sse_stream and it IS applied — to /search/stream. Verified by reading routes.py:795-806 the _rl helper at routes.py:46-51, with control needle confirming other *_limit_decorator symbols show real us in api/init.py so the zero-hit was not a search. The agent's MEASUREMENT (on /theme/stream) was correct and is v makes this a real finding; its MECHAN was not. Both halves are recorded so t next reader does not re-derive the sam wrong cause. WHY IT MATTERS. An unclassified SSE endpoint is the cheapes to pin server resources: each connecti held open, and the default class permi 120/min of them. The declared sse_str class exists precisely because this rout shape needs a tighter bound than ordi GETs. ACCEPTANCE. (a) A decision, recorded, on whether /theme/stream belongs in the sse_stream class or gen warrants the default — this is a policy question, not automatically a bug to pa (b) If it belongs in sse_stream, the dec is applied and a test drives BOTH SSE routes and asserts each returns its INTENDED class limit from x-ratelimit so a future route added without a class caught. (c) A guard that enumerates S shaped routes and fails on any that can</pre>

ATM ID	Type	Status	Severity	Description
BOB-168	Task	Queued	Low	no explicit rate-limit class — the general form, so the third SSE route does not need this. (d) Honest boundary: this does not claim 120/min is exploitable in this deployment; with network_mode host and no reverse proxy every caller shares the 127.0.0.1 bucket, which BOB-111 measures and recorded separately. NOT CLAIMED change made. The limit values were read from headers, never driven to exhaustion; the limiter is per-IP and shared with concurrent agents on this host (§11.4.1.1). WHAT. scripts/run_all_challenges.sh:60 “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist: \$ sed -n ‘66p’ scripts/run_all_challenges.sh “scaling_horizontal_challenge.sh” \$ ls challenges/scripts/scaling_horizontal_challenge.sh ls: cannot access ...: No such file or directory VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation. WHY IT MATTERS. Whether it is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and that is the first thing to determine: if it SKIPS silently, challenge bank advertises coverage it does not have (a §11.4.266 claim-vs-reality mismatch with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which trains readers to ignore it. Neither outcome is acceptable; they need different fixes. ACCEPTANCE. (a) Determine and record the runner’s actual behaviour on a missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history for whether it once existed and was deleted, since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own fault; a missing challenge must be loud (§11.4.124 SKIP-with-reason at minimum), never absent-and-quiet. SEVERITY. Low as a

ATM ID	Type	Status	Severity	Description
BOB-169	Bug	Queued	Medium	defect, but it sits on the challenge-cov seam, so (c) may deserve its own item. WHAT. The §11.4.65 markdown-export mandate requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .h files under docs/ (excluding dist/ and node_modules/) are pandoc FRAGMENTs; they begin at WHAT. The BOB-109 scaling suite gates two timing tests on loadavg_1m/nproc 0.75 — on this 8-cpu host, a 1-minute l average at or below 6.0 — and SKIPs l otherwise. That gate is correct and was adopted for a good reason (§11.4.201(6)); the author validated a 2.10 growth-exp threshold, then observed it FALSE-FAIL; correct code at load 11.06 (baseline sp 2.136), and at load 15-23 measured ba spans of 2.358 against injected-cubic mutants at 2.278-2.331 — the signal is smaller than the noise and the two are separable in either direction. Gating w honest response. THE PROBLEM. The has consequently NEVER RUN under its own precondition. Every recorded run measured load 9.15-23; the independent reviewer measured 9.66 during review; conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at lo 9.4-11.6 (span 1.882) and in three stab runs (1.759/1.832/1.848) — but never beneath the shipped ceiling. The author states this honestly as an inference from data rather than a run they can paste, the reviewer confirmed the inference is sound (quiescence is strictly more favourable). Sound inference is still not observed run. WHY THAT MATTERS (§11.4.226). A registered, topology-pre guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled quiet-window invocation, no tracked item — which is precisely the unexecuted-standing-guard class §11.4 names, where registration gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is standing guards that, when finally executed,
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				immediately emitted latent FAILs. ACCEPTANCE. (a) Execute both quiescent timing tests on a host whose 1-min loadavg is genuinely $\leq 0.75/\text{cpu}$ — no agent fleet, no concurrent build — and capture the run as an artifact showing resolved load reading alongside the measured span exponent, so the precondition is provable from the artifact and not asserted. (b) Record the observed GREEN polarity in the item and in the source, replacing the current honest-inference note. (c) If the gate FAILS upon genuine quiescence, that is the finding item exists to surface, and it reopens the threshold-calibration question rather than being worked around. (d) Establish a freshness contract per §11.4.226: how a quiescent verdict may be before the counts as uncovered again, so this does not silently return to never-executing. HONEST BOUNDARY. This item does not claim the gate is wrong. The evidence points the other way — quiescence is strictly more favourable than the loads where GREEN was already observed. It claims only that the run has not happened, and that an unexecuted guard cannot be cited as coverage. PROVENANCE. Raised as IMPORTANT-2 by the independent Fabric substrate reviewer of the BOB-109 work, and independently corroborated: the reviewer verified the SKIP is loud and its resolved numbers, and confirmed that shipped tests do SKIP at today's load. WHAT. Both rate limiters key their per bucket on the LEFTMOST element of X Forwarded-For when TRUST_FORWARDED_FOR is enabled. download-proxy/src/api/rate_limit.py:117-123 (:7187) and the mstdlib limiter in plugins/download_proxy (:7186), which was deliberately built to exact policy parity with it. That element CLIENT-SUPPLIED. An honest reverse APPENDS the peer address to the RIGHT whatever arrived, so the leftmost entry whatever the client sent — attacker-controlled by construction, not by misconfiguration. Consequences with t
BOB-171	Bug	Queued	Low	

ATM ID	Type	Status	Severity	Description
				flag on: (1) rotating a forged leftmost v mints an unlimited sequence of fresh buckets, which is total bypass of the lin the flag is supposed to make MORE accurate; (2) it also defeats the LRU/id reap cap, since each forged identity is distinct key — the bucket map is a mer growth surface bounded only by the ca and the cap's eviction then discards th budgets of REAL clients to make room forged ones. NOT EXPLOITABLE AS DEPLOYED TODAY, and that is why th Low, not High. TRUST_FORWARDED_ defaults OFF at both sites, and the dep stack runs network_mode host with no reverse proxy in front (verified across five compose services), so peer address are real client IPs and no NAT collapse occurs. The defect is latent: it arms the moment someone puts a proxy in front turns the flag on to recover real client which is exactly the situation the flag e for. The failure mode is therefore 'corr looking configuration change silently disables the limiter', not 'currently bro PROVENANCE. Raised as MINOR-3 by independent reviewer of the BOB-111 limiter work and independently confir by the implementing agent, which note at BOTH source sites as a tracked follo rather than fixing it in that change. Fil here because neither agent has write a to the tracker. WHY IT COVERS BOTH PORTS. :7186's limiter was built to deliberate policy parity with :7187's, including this parsing. Fixing one and the other would leave the two surfaces disagreeing on client identity — worse the shared defect, because it becomes surface-dependent and untestable as a single invariant. ACCEPTANCE. (a) Re leftmost-element parsing with a trust-a resolution at BOTH sites: either rightm minus-N-trusted-hops, or a trusted-pro CIDR allowlist where only a peer insid allowlist may assert XFF at all — the c is a deployment-topology decision and should be recorded, not assumed. (b) A that forges a rotating leftmost element asserts the bucket key does NOT rotat

ATM ID	Type	Status	Severity	Description
BOB-172	Bug	Queued	High	site. (c) Paired §1.1 mutation: restore leftmost parsing and the test must FAIL. Negative control (§11.4.201(1)): with TRUST_FORWARDED_FOR OFF, the header must be ignored entirely and keying must fall back to the peer address — a guard starts honouring XFF when the flag is fixed. (e) Honest boundary: this closes header-forgery bypass; it does not make per-IP limiting fair behind NAT, where real users legitimately share one address. NOT CLAIMED. No change made. Both still parse leftmost; the flag still default WHAT. The rutracker SEARCH endpoint refused by bot protection while the site itself is fully reachable. Measured unauthenticated 2026-08-21 (no cookies read, no credential value in any artifact §11.4.10): GET /forum/index.php -> 200 96283 B, WHAT. download-proxy/src/api/hooks.py:96-102: def _save_hooks(hooks: list[dict[str, Any]]) -> None: try: os.makedirs(os.path.dirname(HOOKS_FILE), exist_ok=True) with open(HOOKS_FILE, 'w') as f: json.dump(hooks, f, indent=2) except Exception as e: logger.error(f'Failed to save hooks: {e}') It catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both sites then report success unconditionally:152 _save_hooks(hooks) :174 _save_hooks(hooks) :154 logger.info('Created hook: ...') :175 logger.info('Deleted hook: ...') :156 return HookResponse(hook_id=..., ...) :176 return {'message': 'Hook deleted', ...} USER: VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 OK, hook_id, and the hook was never written; their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is
BOB-173	Bug	Ready for testing	High	logger.info('Created hook: ...') :175 logger.info('Deleted hook: ...') :156 return HookResponse(hook_id=..., ...) :176 return {'message': 'Hook deleted', ...} USER: VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 OK, hook_id, and the hook was never written; their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is

ATM ID	Type	Status	Severity	Description
				log line nobody is watching. THIS IS T §11.4.252 SHAPE. The path combines dangerous capabilities from that anchored taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks are outbound calls the system will or will not make) — so it is required to FAIL CLOSURE verify the precondition, refuse when it cannot be satisfied, and surface the reason. Instead it fails open, and a bare ‘exception:’ that only logs is the exact pattern §11.4.252 enumerates. At the product layer it is also a §11.4.201(6) failure: null: a successful write and a swallowed failure are indistinguishable to the caller. PROVENANCE. Surfaced as an out-of-context observation by the independent review of the BOB-135 test-isolation work — this swallow is what turned that defect into ‘assert 0 == 1’ mystery, because the EACCES on /config was logged and discarded while the endpoint kept returning 200. Verified here directly from source before filing (the function body and both sites read above), not taken on report. Recorded as a §11.4.238 discovery-chance escape: found by an agent reading code during an unrelated investigation, not automated QA regime — the coverage a defect of equal standing to the defect itself. ACCEPTANCE. (a) _save_hooks propagates failure — raise, or return a status the callers must consume. (b) Both endpoints translate a persistence failure into an HTTP error (500-class), never a success body; a create that did not persist must not return a hook_id. (c) A test drives each endpoint with the hooks file unwritable (read-only dir or a patched open raising OSError) and asserts a non-2xx status that a subsequent GET does not list the phantom hook — assert on the user-observable outcome, not on the log line. Paired §1.1 mutation: restore the swallowed the test must FAIL. (e) Audit the same for sibling swallows — this is a pattern one instance is rarely alone. (f) Honest boundary: this does not claim the write currently fails in production; it claims

ATM ID	Type	Status	Severity	Description
				WHEN it fails the user is told the oppo and the BOB-135 investigation shows i fail in at least one real environment. N CLAIMED. No change made. No assess of how often the write fails in the oper deployment. RECORDING A SHELL ER OF MY OWN (§11.4.6): the first version this description was written with the a pattern snippet inside backticks in a d quoted shell argument, so the shell ran command substitution and the text wa replaced by nothing — the stored description read ‘and is the exact anti-pattern’. This is the SECOND time this session that backticks-inside-double-qu has corrupted content (the first mangl commit message). Fixed here by editin through a Python client with no shell quoting in the path. Noted because a silently-truncated defect description is exactly the kind of quiet corruption §11.4.201(7)(c) warns about — the pat part of the instrument, and it failed wi erroring. WHAT. A corrupt hooks file is silently indistinguishable from “no hooks configured”, and the next create then DESTROYS every existing hook while returning HTTP 200. Three defects on path, filed together because fixing any alone leaves the data loss reachable. A download-proxy/src/api/hooks.py:104-1 _load_hooks wraps the read in except Exception: logger.error(...) and falls through to return []. A truncated or malformed hooks.json is therefore repo to every caller as an empty, healthy ho list. Confirmed at source. A2 — the consequence, and the reason this is Hi create_hook loads, appends, saves. Giv corrupt file that load turns into [], the writes a one-element list over the top. previously-configured hook is gone. Measured by the implementing agent against the ALREADY-FIXED tree, so t survives the BOB-173 write-failure fix: BEFORE on disk : ['prod-hook-0', 'prod hook-1', 'prod-hook-2'] GET reports : 2 {'hooks': [], 'count': 0} <- claims ZERO hooks configured DELETE reports : 40
BOB-174	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				{‘detail’: ‘Hook not found’} <- for a ho that IS in the file POST reports : 200 hook_id=3c8a5930-... AFTER on disk : [‘3c8a5930-...’] Three prod hooks destr HTTP 200 throughout, nothing surface the user. A5 — _save_hooks writes with plain open(path, "w"). A crash, ENOSP kill mid-write truncates the file in plac That is precisely the corruption A1 the reads as “no hooks” and A2 overwrites same codebase already has the correct pattern: theme_state.py:115-126 uses file + os.replace. WHY THE THREE AF ONE ITEM. A5 manufactures the corru file, A1 misreads it as empty, A2 destr the contents. Fixing only A1 leaves truncation reachable; fixing only A5 le an existing corrupt file a data-loss trig fixing only A2 leaves the API lying abo what is configured. The chain is the de DELIBERATELY NOT FIXED UNDER BOB-173, and the reasoning is sound. I implementing agent identified all three while auditing sibling swallows as that required, and declined to fix them in th same change because: they change GE semantics on an endpoint the frontend consumes (200 -> 5xx); they need a CORRUPT-file reproduction rather than UNWRITABLE-file one BOB-173 built; they need a design decision that is genuinely not obvious — a MISSING h file legitimately means “no hooks configured”, while a CORRUPT one do not, and today’s code cannot tell those apart. Expanding BOB-173’s scope to c them would have meant shipping that decision unexamined. ACCEPTANCE. (C Distinguish MISSING from CORRUPT: missing file remains an empty list; a co one is an error, never silently []. (b) De and RECORD what GET does on corru — 5xx, or 200 with an explicit degrade marker the frontend can render. This i design decision, and it should be state rather than inferred from whatever the patch happens to do. (c) create/delete MUST NOT overwrite a file they could parse — refuse, do not clobber (§11.4.2 mutation plus external side effect mus

ATM ID	Type	Status	Severity	Description
BOB-175	Task	Queued	Low	closed). (d) Make the write atomic via os.replace, reusing theme_state.py's existing pattern rather than re-inventing (§11.4.28). (e) Tests asserting the USE OBSERVABLE outcome, not log lines: s corrupt file, assert GET does not claim hooks, assert a create refuses rather than destroying, and assert the file still holds original hooks afterwards. (f) Paired \$1 mutation per guard. (g) NEGATIVE CONTROLS (§11.4.201(1)): a MISSING must still yield an empty list and a word create; a VALID file must behave exactly today. A fix that makes every load fail or by failing always is not a fix. A3, RECORDED SEPARATELY, NOT PART OF THIS CHAIN. VALID_EVENTS and HookEventType are two sources of truth, one closed set. Verified identical today, unguarded against drift: if they diverge, hook registers successfully and then s never fires. One assertion pinning them would close it. NOT CLAIMED. No change made. The BOB-173 write-failure fix is and orthogonal — it makes a FAILED v honest; it does nothing about a SUCCESSFUL write of wrong data derived from a misread file. WHAT. The workable-items update searches guards the status-location invariant in direction only. After BOB-166, update -status <terminal> on an Issues-located refused. The mirror is still open: update location Fixed --status <non-terminal> 0 and creates a row that update's own validator immediately rejects. Verified empirically by the independent review BOB-166: update -location Fixed -status progress' -> exit 0 validate -> FAILS, naming the row (check (f), fixedLocationNonTerminalStatus) So the command can still mint a state its own validate call refuses. That is the §11.4.196(F) shape at the seam layer: invariant is CONFIGURED in validate, not ENFORCED at every write path that violate it. WHY IT WAS DELIBERATELY LEFT OPEN IN BOB-166, and why that's right. Three reasons, all checked rather than asserted: (1) the real corpus has 2

ATM ID	Type	Status	Severity	Description
				instances of this direction against TEN the direction BOB-166 closed — the asymmetry in the data matched the asymmetry in the code; (2) detective coverage ALREADY existed and demonstrably fires, so unlike BOB-166 direction this cannot rot silently; (3) an load-bearing one — reopenCmd documented and SANCTIONS a transient Fixed-plus terminal state via its --location Fixed operator override. A naive preventive guard at the update seam would collide with reopenCmd's semantics. Closing this then requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined. ACCEPTANCE. (a) Decide and RECORD a preventive guard on this direction collides with reopenCmd's sanctioned override; this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidate worth weighing: exempt the reopen path explicitly, require an override flag on reopen mirroring reopen's, or leave the direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it requires terminalStatuses() — BOB-166 establishes one predicate source specifically so the directions cannot drift. (c) Paired §1.1 mutation. (d) NEGATIVE CONTROL (§11.4.201(1)), and it is the sharp one in the sanctioned reopen path MUST still work. A guard that breaks reopenCmd documented override is a false-positive refusal, and worse than the gap it closes because it breaks a workflow operator told to use. HONEST BOUNDARY. This is not a live data defect. Zero corpus instances, and the detective gate catches at the next validate. It is filed so a scope deliberate omission stays visible as a tracked decision instead of living only in code comment where the next reader will mistake it for an oversight (§11.4.197: started thread reaches a documented terminal state, never quiet abandonment).

ATM ID	Type	Status	Severity	Description
				PROVENANCE. Raised by the implementer of BOB-166 as a known asymmetry. It was deliberately not closed, independently verified and endorsed as an acceptable scope boundary by that item's reviewer. The condition that it become a tracked item rather than a comment. This is that item